

DIBRIS

DEPARTMENT OF INFORMATICS,
BIOENGINEERING, ROBOTICS AND SYSTEM ENGINEERING

BIOMEDICAL ROBOTICS 2026/27

Assignment 1

Author:

Antonio Mazzotta - 8973098
Francesco Sisani - 8571674
Matteo Spluga - 9185902

Professor:

Maura Casadio
Federica Sassi

Contents

0.1 MCI 2

1 Part 1 **2**
1.1 Data organization 2
1.2 Part 1 solution 2
1.2.1 Preprocessing steps for each muscle 3
1.2.2 Comparison between the reconstructed and MCI registered envelopes 3

2 Part 2: MCI Mapping and Cursor Velocity Computation **5**

3 Part 3: Cursor reconstruction and position tracking **5**
3.1 Reaching Analysis Code Overview 5

4 Part 4: Performance per reach **5**

Introduction

This first assignment of the course consists in steering a cursor with four different muscles. The group was requested to choose a person who would have worn the MCI to record a reach session. This was composed of a setup and calibration phase and a play part. In conclusion, once the group had the data, the main objective was to rebuild the cursor from the raw EMG data and compare them.

More in detail, the assignment can be divided in 3 parts:

- Part 1: Band-passing the raw EMG signal in a range of 30-450Hz, then rectifying it and applying a low-pass filter at 5Hz. This is done to obtain envelopes.
- Part 2: Using the gains and the envelope to create the map that goes from muscle intensity to cursor velocity.
- Part 3: Integrating the cursor velocity into a position and animating it with the targets. It is requested to compare it with the logged cursor.
- Bonus part: Using our cursor to drive a simulated 2-link robot arm.

MCI

An MCI (Myoelectric Computer Interface) system is a hardware and software architecture used for neuro-rehabilitation purposes. The system acquires electric activity from the muscles (EMG signals, which stands for ElectroMyoGraphic signals) via surface sensors placed on the patient's limbs. The signals are elaborated in real time and projected on a display. The patient uses this visual biofeedback to correct their incorrect muscular activation schemes to help their brain re-label correct movements.

1 Part 1

1.1 Data organization

In this subsection relative to Part 1 of the assignment, there is a brief explanation of the initial phase of the assignment itself, which consists of three main steps:

- Generation and loading of the file $Group_6.mat$
- Data selection
- Information extraction from the table

After having defined the path to find the session registration data (`ID_6_Session_1`), the `make_group_mat` function collects and organizes the original data acquired from the MCI, generating the `Group_6.mat` file. This file contains the EMG signals, action envelopes, cursor positions, reached targets and calibration information. Then, based on the variable "task" value, the corresponding "center-out" or "random" experiment registrations are selected. Here, the focus was associated to the center-out case. Center-out or random tables are stored in a matrix called "`D`" and the corresponding targets are stored in a matrix simply called "`targets`". Finally, necessary data for the elaborations required for Part 1 are extracted from matrix `D`. In the last part, raw EMG signals (expressed in ADC counts) are converted in microvolts.

1.2 Part 1 solution

Part 1 regards the preprocessing of raw EMG signals, allowing to get the activation envelopes starting from raw EMG signals and to compare the results with the ones produced by the MCI system by terms of correlation and temporal difference. The process follows four steps:

- Band-pass filtering
- Rectificaton
- Low-pass filtering
- Normalization

1.2.1 Preprocessing steps for each muscle

The first initial step allows to transform the raw EMG signals in a measurement of the muscular activation. Initially, a band-pass filter (between 30Hz and 450Hz) is applied to eliminate unwanted low-frequency and high-frequency components. Here, a fourth-order Butterworth filter is projected and the DC offset of each EMG signal is also removed. The filter is applied with the function `emgFilt`. Then, the signals gets rectified with the `emgRect` function, making its values positive, and filtered with a fourth-order Butterworth low-pass filter at 5Hz to obtain a more regular trend. This result is called activation envelope and it represents the intensity of muscular activation in time. It is stored in `envOursUV`. Finally, a normalization of these is carried out with respect to the maximum calibration value for each muscle. With this process, values between 0 and 1 are obtained so that the activations of the four muscles are comparable.

Here the plotted signals are shown for the four muscles:

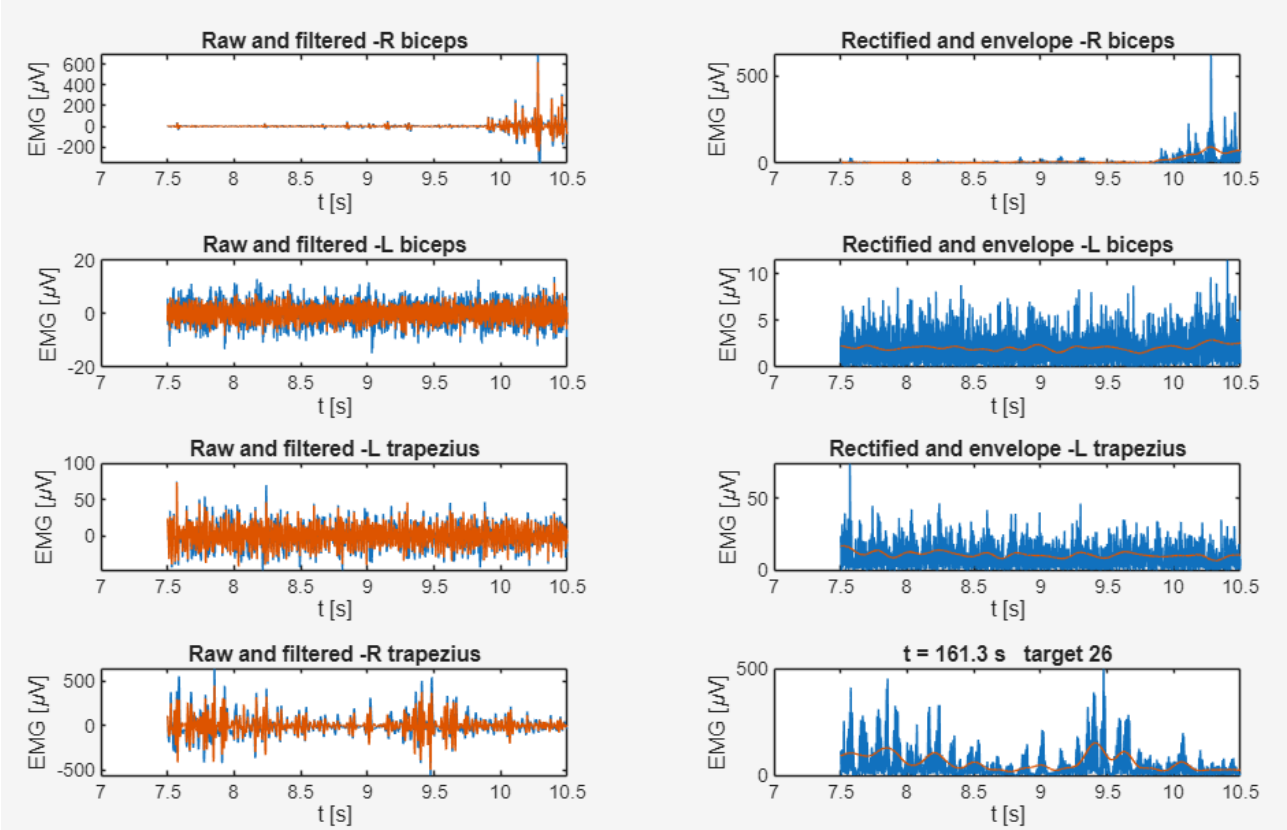


Figure 1: EMG signals processed with the filtering, rectification and DC values elimination processes.
Left images: blue → raw signals, red → filtered signals.
Right images: blue → rectified signals, red → envelopes.

1.2.2 Comparison between the reconstructed and MCI registered envelopes

Then, in the second part, a quantitative comparison is performed between the reconstructed envelopes `envOursNorm` and the ones registered by the MCI system `envMCI`. The correlation matrix between the two envelopes is computed for each muscle to assess how the respective trends are similar. Then, the cross-correlation between signals is computed. With this one, the corresponding delay at the maximum value of the cross-correlation is identified, and it is converted from samples to milliseconds.

Here, the results are shown from the Command Window:

```
Part 1: Correlation matrices
  Muscolo R biceps:
    1.0000    0.8330
    0.8330    1.0000

  Muscolo L biceps:
    1.0000    0.8891
    0.8891    1.0000

  Muscolo L trapezius:
    1.0000    0.8294
    0.8294    1.0000

  Muscolo R trapezius:
    1.0000    0.8593
    0.8593    1.0000
```

Figure 2: Correlation matrices for each muscle

```
Part 1: Delay values for each muscle

Muscolo R biceps: delay =     -40


Muscolo L biceps: delay =    -38.5000


Muscolo L trapezius: delay =    -40.5000


Muscolo R trapezius: delay =    -40.5000
```

Figure 3: Delay values from cross-correlation for each muscle

Subsequently, the two envelopes are overlapped, using a time window of 30 seconds, to allow a visual comparison:

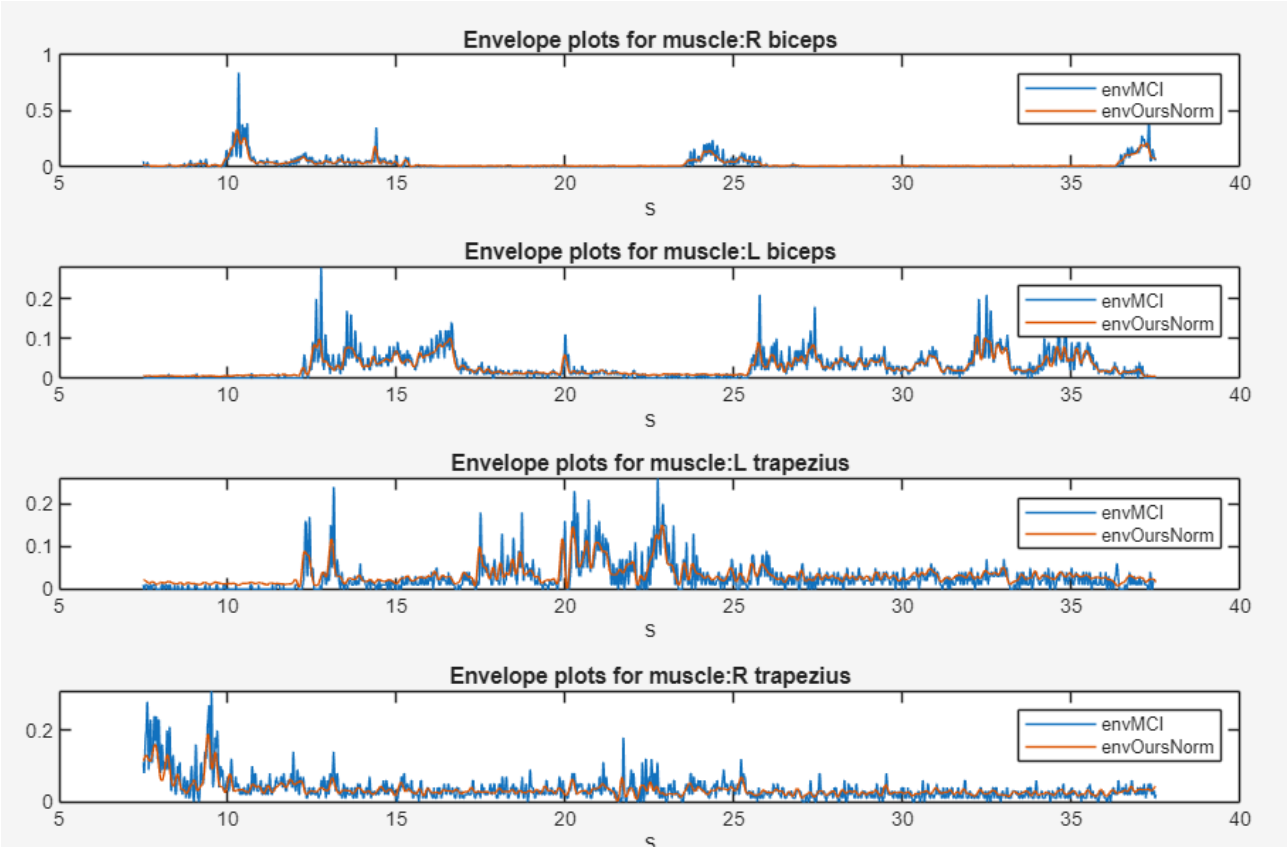


Figure 4: Overlapped MCI and reconstructed envelopes

At the end, signals are converted from a sampling frequency of 2000Hz to 100Hz. The downsampling is done only after the filtering and the envelopes computation, since raw EMG signals contain high frequency components that would be lost or altered reducing beforehand the sampling frequency.

2 Part 2: MCI Mapping and Cursor Velocity Computation

The second part of the assignment consists of the computation of the cursor velocity starting from the activation envelopes of the four muscles, using the mapping defined by the MCI system.
The MCI system uses velocity control, in which the level of muscle activation determines the cursor velocity rather than its position. Each axis is controlled by a pair of muscles: the biceps regulate horizontal movement (X), while the trapezius muscles regulate vertical movement (Y). The velocity along each axis is proportional to the difference between the activation levels of the two associated muscles, so that their simultaneous contraction cancels each other out. The gain, equal to 50 cm/s, determines the maximum cursor velocity when one muscle is activated at 100% and its antagonist is relaxed. These relationships are described by the mapping matrix (W), which allows the velocity components to be calculated from the activation envelopes of the four muscles.
Here the calculations that will determine the speed along each axis is reportedly shown:

```
 Part 2 - MCI map:
x speed = 50 * (R biceps - L biceps)
y speed (down) = 50 * (R trapezius - L trapezius)
```

Then, a dead zone of 5% is applied to the activation envelopes (`envOursD` and `envMCID`). Every activation value below 0.05 is put equal to 0. This procedure allows to eliminate small residual activations caused by noise, avoiding unwanted cursor movements when muscles are at rest.

At last, the cursor velocities are computed using the modified activation envelopes, by applying the MCI map to the two envelopes matrices.

- `velOurs` is the cursor velocity computed using the envelopes reconstructed in Part 1.
- `velMCI` is the cursor velocity computed using the envelopes registered by the MCI system.

3 Part 3: Cursor reconstruction and position tracking

3.1 Reaching Analysis Code Overview

This part performs a reaching analysis by simulating the movement of a cursor based on velocity data and comparing it to a desired cursor position. The code initializes position arrays for two methods: one for the custom algorithm (denoted as 'ours') and one for the Minimum Jerk Control (MCI) method. It then iterates through time steps, updating the positions based on velocity and applying clipping controls to ensure the positions stay within the screen boundaries. If a reset condition is met, the positions are reset to the desired cursor position at the current time step. Finally, the code calculates the error between the simulated positions and the desired positions for both methods and prints the mean cursor error for each method.

4 Part 4: Performance per reach

This part performs a reaching analysis on a set of targets. It first identifies targets that have been hit and are outward from a central point. It then extracts the reach times and overshoots for these selected targets. Finally, it plots the reach times and overshoots in two separate subplots for visualization.